

CSCE 465/765: Computer & Network Security

Homework Assignment 2

Protect Agent Messages with Classic Cryptography

Deepti Patil

October 7, 2026

Task 0 - Lab Preparation

I completed the work inside the course VM using a Python virtual environment. The environment used Python 3.12.3, OpenSSL 3.0.13, cryptography 49.0.0, and pytest 9.1.1. I also generated the standardized ffdhe3072 parameter file required for the Diffie-Hellman portion of the assignment. The screenshot below records the required environment information with the virtual environment active.

```
GROUP: ffdhe3072
(.env) deepti@csce465:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3
(.env) deepti@csce465:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.env) deepti@csce465:~/csce465-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/deepti/csce465-agentsec/hw2/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/deepti/csce465-agentsec/hw2/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.env) deepti@csce465:~/csce465-agentsec/hw2$
```

Figure 1. Lab environment versions for Python, OpenSSL, cryptography, and pytest.

Task 1 - Why Encryption Alone Is Insufficient

Design. I implemented `baseline_ctr.py` using AES in CTR mode to encrypt the fixed command `{"action": "READ", "path": "notes.txt"}` without a MAC. The relay does not know the encryption key. Instead, it uses the XOR difference between the bytes for READ and the equal-length replacement EDIT and applies that difference to the corresponding ciphertext bytes.

Experiment. The modified ciphertext decrypted successfully as an EDIT command even though the relay never learned the key. The XOR differences were 0x17, 0x01, 0x08, and 0x10 for the four changed bytes. I then delivered the same original ciphertext twice, and the receiver processed the READ command both times.

Root cause. CTR mode provides confidentiality but is malleable when it is used without authentication. Because CTR effectively XORs plaintext with a keystream, a controlled change to ciphertext produces a corresponding change in decrypted plaintext. In addition, encryption by itself does not provide freshness or state, so the receiver cannot distinguish a replayed ciphertext from a new one. Integrity protection and replay tracking are therefore required in addition to confidentiality.

```
(.venv) deepiti@csce465:~/csce465-agentsec/hw2$ nano baseline_ctr.py
(.venv) deepiti@csce465:~/csce465-agentsec/hw2$ python baseline_ctr.py
Original plaintext:
{"action": "READ", "path": "notes.txt"}

Original ciphertext (hex):
a8e940393d5c614aa17e2815d3cbde127df218d5f5d0be54dc6cbcb265a8f03ddee51245

XOR relation used by relay:
Original bytes:    b'READ'
Replacement bytes: b'EDIT'
byte 0: 0x52 XOR 0x45 = 0x17
byte 1: 0x45 XOR 0x44 = 0x01
byte 2: 0x41 XOR 0x49 = 0x08
byte 3: 0x44 XOR 0x54 = 0x10

Modified ciphertext (hex):
a8e940393d5c614aa17e2802d2c3ce127df218d5f5d0be54dc6cbcb265a8f03ddee51245

Receiver decrypts modified ciphertext:
Receiver processed: {"action": "EDIT", "path": "notes.txt"}

Replay demonstration:
First delivery:
Receiver processed: {"action": "READ", "path": "notes.txt"}
Second delivery of SAME ciphertext:
Receiver processed: {"action": "READ", "path": "notes.txt"}

Result:
The receiver processed the same encrypted command twice.
AES-CTR encryption alone provides no integrity or replay protection.
(.venv) deepiti@csce465:~/csce465-agentsec/hw2$
```

Figure 2. AES-CTR bit-flip and replay demonstration. READ is changed to EDIT, and the same ciphertext is accepted twice.

Task 2 - Authenticated Finite-Field Diffie-Hellman Setup

I implemented an authenticated in-process handshake between a simulated gateway and node. Each side uses a long-term 3072-bit RSA signing key with RSA-PSS and SHA-256. For each session, fresh ephemeral FFDHE3072 keys and fresh nonces are generated. The handshake builds a canonical, length-prefixed transcript containing the protocol label, group identifier, both identities, both ephemeral public values, and both nonces. Each endpoint signs its role together with the SHA-256 transcript hash and verifies the peer signature before accepting the session.

Diffie-Hellman provides key establishment because both endpoints independently derive the same shared secret without sending that secret directly. The RSA-PSS signatures provide entity authentication by binding the exchange to the long-term identities. The fresh nonces and ephemeral DH values make sessions fresh and help prevent replay. Because new ephemeral DH private values are generated for every session, compromise of a long-term signing key later does not directly reveal previously established DH secrets, providing forward-secrecy properties for the exchanged session keys.

The transcript hash and DH secret are used in the assignment-specific derivation to produce a master key, separate gateway-to-node encryption and MAC keys, separate node-to-gateway encryption and MAC keys, and a session identifier. The distinct labels create key separation so that one key is not reused for multiple

cryptographic purposes or directions. My output confirms that the gateway and node derived the same DH secret and that the directional keys were distinct.

Signing the identities, roles, nonces, and ephemeral DH public values binds all important handshake fields together. An on-path attacker cannot replace a DH value or nonce without causing signature verification to fail. Including identities and roles also prevents a valid message from one participant or role from simply being reflected back as if it came from the peer. If signatures were removed, an active attacker could substitute its own ephemeral DH values, establish separate secrets with the gateway and node, and act as a man-in-the-middle while relaying or modifying traffic.

```
(.venv) deepiti@csce465:~/csce465-agentsec/hw2$ python3 --version
python3.12.3
openssl version
openssl 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
pip show cryptography pytest
Name: cryptography
Version: 40.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/deepti/csce465-agentsec/hw2/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 8.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/deepti/csce465-agentsec/hw2/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) deepiti@csce465:~/csce465-agentsec/hw2$
```

Figure 3. Successful authenticated FFDHE3072 handshake, transcript authentication, session derivation, and key-separation checks.

```

Required by:
(.venv) deepiti@csce465:~/csce465-agentsec/hw2$ python baseline_ctr.py
Original plaintext:
{"action": "READ", "path": "notes.txt"}

Original ciphertext (hex):
4cb4c635bd799195b24516bffa1064ce9bc13512de07540097e93de373742fccc693936fa

XOR relation used by relay:
Original bytes:  b'READ'
Replacement bytes: b'EDIT'
byte 0: 0x52 XOR 0x45 = 0x17
byte 1: 0x45 XOR 0x44 = 0x01
byte 2: 0x41 XOR 0x49 = 0x08
byte 3: 0x44 XOR 0x54 = 0x10

Modified ciphertext (hex):
4cb4c635bd799195b24516a8f00e5ce9bc13512de07540097e93de373742fccc693936fa

Receiver decrypts modified ciphertext:
Receiver processed: {"action": "EDIT", "path": "notes.txt"}

Replay demonstration:
First delivery:
Receiver processed: {"action": "READ", "path": "notes.txt"}
Second delivery of SAME ciphertext:
Receiver processed: {"action": "READ", "path": "notes.txt"}

Result:
The receiver processed the same encrypted command twice.
AES-CTR encryption alone provides no integrity or replay protection.
(.venv) deepiti@csce465:~/csce465-agentsec/hw2$

```

Figure 4. Handshake adversarial tests: valid signature succeeds while changed transcript, wrong RSA key, and reflected role cases are rejected.

Task 3 - Encrypt-then-MAC Record Layer

I implemented `secure_record.py` with `seal()` and `open_record()` for an encrypt-then-MAC record layer. Each record contains the required version, direction, sequence number, message type, and ciphertext length header. The IV is formed from the 8-byte session ID and the 8-byte sequence number. The plaintext is encrypted with AES-256-CTR, and HMAC-SHA-256 authenticates the header, IV, and ciphertext.

The implementation keeps independent encryption and MAC keys for each direction and starts sequence numbers at zero. A sender therefore gets a different IV for every record under a given encryption key. On receipt, the exact next sequence number and expected direction are checked. The HMAC is verified before any plaintext is released. Modified records, wrong-direction records, and replayed records are rejected rather than partially processed.

The demonstration below shows a gateway-to-node record being authenticated and decrypted correctly. When the same record is submitted again after the expected sequence number has advanced, it is rejected as an unexpected or replayed sequence number.

```

AES-GCM encryption alone provides no integrity or replay protection.
(.venv) deepiti@csce465:~/csce465-agentsec/hw2$ python handshake.py
Loading FFDHE3072 parameters...
Generating long-term 3072-bit RSA signing keys...
Generating fresh ephemeral FFDHE3072 keys...
Signing canonical transcript...
RSA-PSS signatures verified.
Gateway and node derived same DH secret: True

Handshake successful.
Protocol: CSCE465-HS-v2
Group: ffdhe3072
Gateway nonce: a9fa0a76975ba936c4fb2d0ec21d8114
Node nonce: 6f820fe2ef2faae1d2fbe84527d14936
Transcript hash: efa4854c0dfea8f4d06c6842757ad2b48669b7a507ad7ebd801f6fdaf6b69529
Session ID: 44767847f923462a

Derived directional keys:
K_g2n_enc: 1626086d3b27e5193d6acad63eae1fa0ed6327f0b03a8c5a1fba5ddd65986c63
K_g2n_mac: b7f26ce6c53320d6fe62f2b40392d475ba956f6114a7c85074df8680ad6afcfe
K_n2g_enc: f13ee7b72a592694b5534a02523d2681b880842a20959754c0a353eca9db0e38
K_n2g_mac: f88ec99fa2792e0a34f24a8eea4da4e4ee26932063a344cbb6f8998018f1affa

Key separation checks:
G2N encryption != G2N MAC: True
G2N encryption != N2G encryption: True
(.venv) deepiti@csce465:~/csce465-agentsec/hw2$

```

Figure 5. Valid protected record is authenticated and recovered; replay of the record is rejected.

```

(.venv) deepiti@csce465:~/csce465-agentsec/hw2$ pytest -v test_handshake.py
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/deepiti/csce465-agentsec/hw2/
.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/deepiti/csce465-agentsec/hw2
collected 4 items

test_handshake.py::test_valid_signature PASSED [ 25%]
test_handshake.py::test_changed_transcript_rejected PASSED [ 50%]
test_handshake.py::test_wrong_rsa_key_rejected PASSED [ 75%]
test_handshake.py::test_reflected_role_rejected PASSED [100%]

===== 4 passed in 6.24s =====
(.venv) deepiti@csce465:~/csce465-agentsec/hw2$

```

Figure 6. Record-layer tests verify valid processing and rejection of replay, wrong direction, ciphertext tampering, and tag tampering.

Task 4 - Adversarial Tests

I used pytest to exercise both successful behavior and specific safe failures. The tests cover successful authenticated handshakes, transcript modification, use of the wrong RSA key, reflected roles, valid protected records, replayed records, wrong-direction records, modified ciphertext, and modified authentication tags. In each rejection case, the operation fails rather than releasing unauthenticated plaintext. The screenshots in Figures 4 and 6 show the handshake and record-layer test suites passing.

Security Note

The attacker model for this design is an active on-path attacker who can observe, copy, delay, replay, reorder, and modify messages exchanged between the gateway and node. The attacker may try to change an encrypted command, replay a previously valid record, reflect a message into the opposite direction, or

interfere with the handshake. The design assumes that the endpoints themselves and their long-term private signing keys are trusted. Endpoint compromise, malicious application logic, unsafe tool authorization, denial of service, and protection of a host after an attacker already controls it are non-goals. Cryptography protects the communication channel, but it does not decide whether an authenticated command is appropriate or safe.

AES-CTR requires a unique IV for every message encrypted under the same key. CTR mode turns a block cipher into a stream-like construction by generating a keystream and XORing that keystream with the plaintext. If the same key and IV are reused, the same keystream is reused. An attacker can then XOR the two ciphertexts to cancel the keystream and obtain the XOR of the two plaintexts, which can reveal information about both messages. In this record layer, the IV combines the session ID with the sequence number. Because the sequence number increases for each message in a direction, the sender does not reuse an IV under the same directional encryption key.

Verify-before-decrypt is important because ciphertext should not be treated as trustworthy until its authentication tag has been checked. The receiver first verifies the HMAC over the header, IV, and ciphertext. Only after successful verification does it decrypt and release the plaintext. This prevents modified data from reaching later application logic and gives a clear fail-closed behavior. It also means that changes to security-relevant header fields, such as direction or sequence number, are detected by authentication rather than being silently accepted.

Independent encryption and MAC keys are used for key separation. Encryption and authentication are different cryptographic purposes, so using separately derived keys avoids unnecessary interaction between those purposes and limits the effect of a key being used incorrectly. This design goes further by deriving different encryption and MAC keys for gateway-to-node and node-to-gateway traffic. Directional separation helps prevent reflection and prevents the same key/IV space from being shared by both sides.

The construction is an encrypt-then-MAC design: AES-CTR provides confidentiality, while HMAC-SHA-256 provides integrity and authentication of the protected record. An AEAD mode such as AES-GCM combines encryption and authentication into one standardized operation. GCM can therefore provide confidentiality and integrity with a simpler interface and less opportunity to accidentally omit authentication or verify fields in the wrong order. Both approaches still require correct nonce or IV management. The assignment's explicit encrypt-then-MAC construction is useful because it makes the separate roles of encryption, authentication, sequence checking, and key separation visible.

Finally, a cryptographically valid record is not automatically a safe command. HMAC verification only establishes that the record was created by a party possessing the correct session key and that the protected bytes were not changed in transit. A legitimate authenticated endpoint could still request a dangerous action, or compromised application logic could generate a harmful but correctly authenticated tool call. Authorization and policy checks must therefore occur above the cryptographic record layer. The secure channel can establish who sent a message and whether it was altered, but it cannot determine whether the requested action should be allowed.

Conclusion

The experiments show the progression from confidentiality-only encryption to an authenticated protocol. The baseline AES-CTR example demonstrates that encryption alone does not prevent bit manipulation or replay. The authenticated Diffie-Hellman handshake binds fresh session material to known identities, while the encrypt-then-MAC record layer adds integrity, direction checking, and replay protection. The adversarial tests confirm that expected valid traffic succeeds and that the tested tampering, replay, reflection, and authentication failures are rejected safely.